



— GAME DEVELOPMENT —

OPERAÇÃO RESGATE

ESTRATÉGIA • AÇÃO • MISSÃO



NOME: Jônata Silva Pinho

RA: 115533

UNIFECAP

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas
Disciplina de Game Development

JÔNATA SILVA PINHO

RA 115533

Relatório teórico do desenvolvimento de um jogo de plataforma 2D na Unity

Relatório teórico apresentado à disciplina de Game Development do Centro Universitário UniFECAP, como um dos entregáveis obrigatórios do trabalho “Jogo de Plataforma 2D na Unity”, com o objetivo de evidenciar os fundamentos teóricos, o planejamento e as decisões técnicas aplicadas ao projeto.

RESUMO

Este relatório apresenta os fundamentos teóricos e as decisões de engenharia aplicadas no desenvolvimento de **Operação Resgate**, jogo de plataforma de ação em perspectiva 2.5D construído na engine Unity com programação em C#. O documento aborda os princípios do gênero de plataforma e as técnicas de *game feel* que sustentam um controle responsivo (coyote time, jump buffering e aceleração com inércia); o planejamento do projeto e o conjunto de mecânicas implementadas, incluindo arsenal de quatro armas com balística e mira em 360° e um companheiro de combate com inteligência artificial e progressão; o design de cinco níveis com dificuldade crescente descrito por uma camada de dados; os sistemas de interatividade, HUD e feedback audiovisual; o pipeline de sprites e a máquina de estados de animação; a estrutura do game loop e a arquitetura de componentes desacoplados por eventos; e as estratégias de trilha sonora e efeitos. Ao final, discute-se a conexão entre a teoria estudada na disciplina e os problemas reais de implementação enfrentados e resolvidos, demonstrando que o projeto cumpre integralmente os requisitos propostos e se qualifica como peça de portfólio profissional.

Palavras-chave: Unity; C#; jogo de plataforma; game design; game feel; 2.5D; arquitetura de software.

INTRODUÇÃO

Este relatório apresenta os fundamentos teóricos e as decisões de engenharia aplicados ao desenvolvimento de **Operação Resgate (Operation Rescue)**, um jogo de plataforma e ação em **2.5D**, desenvolvido utilizando a **engine Unity** e programado em **C#**. O documento aborda os princípios do gênero de jogos de plataforma e as técnicas de game feel que proporcionam controles responsivos, como **coyote time**, **jump buffering** e **aceleração inercial**. Também apresenta o planejamento do projeto e o conjunto completo das mecânicas implementadas, incluindo um arsenal de quatro armas orientado por dados (data-driven), com balística e sistema de mira em **360°**, além de um companheiro de combate controlado por inteligência artificial, com seu próprio sistema de progressão.

O relatório descreve ainda o design de cinco fases com dificuldade progressiva, definidas por uma camada declarativa de dados; os sistemas de interatividade, interface do usuário (HUD) e feedback audiovisual; o pipeline de sprites e a máquina de estados de animação; a estrutura do ciclo principal do jogo (game loop) e a arquitetura baseada em componentes desacoplados por eventos; bem como as estratégias adotadas para trilha sonora e efeitos sonoros.

Por fim, discute a relação entre a fundamentação teórica estudada ao longo do curso e os desafios práticos encontrados durante o desenvolvimento, demonstrando como esses problemas foram solucionados na implementação real. O trabalho evidencia que o projeto atende integralmente aos requisitos propostos e possui qualidade suficiente para compor um portfólio profissional de desenvolvimento de jogos.

Palavras-chave: Unity; C#; jogo de plataforma; design de jogos; game feel; 2.5D; arquitetura de software.

LISTA DE ILUSTRAÇÕES

Figura 1 – O game loop da Unity aplicado ao projeto	18
Figura 2 – Arquitetura de módulos e barramento de eventos	19

LISTA DE QUADROS E TABELAS

Quadro 1 – Rastreabilidade entre os requisitos do enunciado e o projeto	10
Tabela 1 – Fichas de atributos das quatro armas do jogador	10
Quadro 2 – Mapa de controles do jogador	11
Quadro 3 – Etapas da metodologia iterativa de desenvolvimento	12
Quadro 4 – Progressão de dificuldade das cinco fases	13
Tabela 2 – Métricas de dimensionamento do level design	14
Quadro 5 – Módulos do código-fonte e responsabilidades	19

LISTA DE ABREVIATURAS E SIGLAS

ABNT – Associação Brasileira de Normas Técnicas

API – Application Programming Interface (interface de programação de aplicações)

GC – Garbage Collector (coletor de lixo de memória)

GDD – Game Design Document (documento de design do jogo)

HDR – High Dynamic Range (alto alcance dinâmico)

HUD – Heads-Up Display (interface sobreposta ao jogo)

IA – Inteligência Artificial

LED – Light-Emitting Diode (diodo emissor de luz)

POO – Programação Orientada a Objetos

RA – Registro Acadêmico

XP – Experience Points (pontos de experiência)

SUMÁRIO

1 INTRODUÇÃO.....	7
2 PRINCÍPIOS DE JOGOS DE PLATAFORMA 2D.....	7
3 PLANEJAMENTO DO PROJETO E MECÂNICAS PROPOSTAS.....	9
4 DESIGN DE NÍVEIS COM PROGRESSÃO DE DIFICULDADE.....	12
5 INTERATIVIDADE, HUD E FEEDBACK.....	14
6 ASSETS, SPRITES E ANIMAÇÕES NA UNITY	16
7 LÓGICA DE PROGRAMAÇÃO EM C# E O GAME LOOP	17
8 ESTRATÉGIAS DE ÁUDIO E EFEITOS SONOROS	20
9 CONEXÃO ENTRE TEORIA E PRÁTICA.....	21
10 CONSIDERAÇÕES FINAIS	22
REFERÊNCIAS	23

1

INTRODUÇÃO

Os jogos de plataforma estão entre os gêneros mais tradicionais e formativos da história dos videogames. Consolidado nos anos 1980 e 1990, o gênero estabeleceu um vocabulário de design – salto, gravidade, obstáculos, coletáveis e fases de dificuldade crescente – que permanece relevante e constitui um laboratório completo para o aprendizado de desenvolvimento de jogos: exige física, animação, design de níveis, interface, áudio e, sobretudo, sensibilidade para a experiência do jogador.

Operação Resgate é um jogo de plataforma de ação militar desenvolvido na Unity como entregável da disciplina de Game Development. O jogador controla um soldado de elite enviado a uma cidade devastada por um desastre radioativo, com a missão de atravessar territórios dominados por criaturas mutantes e forças hostis até concluir a operação de resgate que dá nome ao jogo. Ao seu lado atua o **K9-CYBER ALPHA**, um pastor-alemão cibernético com inteligência artificial própria, que segue o parceiro, combate inimigos, cura o soldado ferido e evolui de nível ao longo da campanha – um diferencial de design que adiciona profundidade tática e vínculo emocional à experiência.

A narrativa é deliberadamente direta, cumprindo o papel previsto no desafio: dar contexto ao personagem e ao universo. Os três pilares que sintetizam a identidade do projeto – **estratégia, ação e missão** – orientaram todas as decisões de design descritas neste relatório.

O objetivo deste documento é apresentar os fundamentos teóricos aplicados no desenvolvimento, conectando cada conceito estudado na disciplina às decisões concretas de implementação. As seções seguem a ordem dos tópicos exigidos no enunciado do trabalho: princípios do gênero (Seção 2), planejamento e mecânicas (Seção 3), design de níveis (Seção 4), interatividade e HUD (Seção 5), assets e animações (Seção 6), programação e game loop (Seção 7), áudio (Seção 8) e a síntese entre teoria e prática (Seção 9). Complementam o documento as listas pré-textuais, as referências elaboradas conforme a ABNT NBR 6023:2018 e o Apêndice A, com a organização de diretórios do projeto.

2

PRINCÍPIOS DE JOGOS DE PLATAFORMA 2D

2.1 O verbo central: o salto

O gênero de plataforma é definido pela navegação de um personagem por superfícies em diferentes alturas, na qual o salto é o verbo central da jogabilidade. Toda a física do personagem – velocidade horizontal, força de pulo e gravidade – forma um contrato com o jogador: se esses valores forem constantes e legíveis, o jogador aprende a julgar distâncias e passa a executar manobras com precisão crescente. Em Operação Resgate, a gravidade é aplicada de forma customizada e constante (26 unidades/s^2), e as forças de pulo e duplo pulo foram calibradas para um alcance máximo de aproximadamente 3,2 unidades de altura, valor usado como régua para dimensionar todas as plataformas e obstáculos do jogo.

2.2 Game feel: responsividade e tolerância ao erro

A literatura de game design descreve como *game feel* a sensação tátil do controle – o que separa um personagem "pesado e travado" de um "vivo nas mãos" (SWINK, 2009). O projeto aplica as três técnicas canônicas do gênero:

- **Coyote time:** uma janela de 0,08 s em que o pulo ainda é aceito após o personagem deixar a borda de uma plataforma. Sem ela, o jogador sente que o jogo "roubou" um pulo que, na sua percepção, foi apertado a tempo.
- **Jump buffering:** o toque de pulo é memorizado por 0,12 s e consumido pelo passo de física seguinte. Isso elimina pulos perdidos por diferença de relógio entre a renderização e a física – um problema real diagnosticado e corrigido durante o desenvolvimento.
- **Aceleração com inércia:** em vez de velocidade instantânea, o personagem acelera e freia com taxas distintas no chão (75 u/s^2 de frenagem, transmitindo pés que "agarram" o solo) e no ar (35% do controle, preservando o comprometimento do salto).

2.3 Curva de aprendizado e recompensa

Dois princípios completam a fundação teórica. Primeiro, a **curva de aprendizado**: cada habilidade deve ser introduzida em contexto seguro, exercitada isoladamente e só então combinada com outras em desafios compostos – princípio que estrutura a progressão das cinco fases (Seção 4). Segundo, a **recompensa pela exploração**: coletáveis guiam o olhar, sinalizam rotas possíveis e premiam o domínio do movimento, transformando o cenário em um convite permanente à maestria.

2.4 A escolha da perspectiva 2.5D

Operação Resgate adota apresentação **2.5D**: a lógica do jogo é estritamente bidimensional – o personagem está preso ao plano XY por restrições do corpo rígido – mas a cena utiliza câmera em perspectiva, iluminação dinâmica em tempo real e paralaxe de fundo em profundidade. A escolha entrega aspecto cinematográfico e moderno sem sacrificar a clareza espacial que o gênero exige: o jogador nunca tem dúvida sobre onde pode pisar, mas o mundo ao redor tem volume, luz e atmosfera.

2.5 Referências de mercado: soluções existentes no gênero

Conforme orienta o roteiro do estudante – reconhecer o que já existe no mercado antes de resolver o desafio –, o projeto estudou soluções consolidadas do subgênero de plataforma de ação com armas (*run and gun*). A série **Metal Slug** (SNK, 1996) é a referência direta de estrutura: fases lineares de avanço lateral, tiro em múltiplas direções, granadas como recurso limitado, veículos e chefes de fim de fase – elementos reinterpretados em Operação Resgate no arsenal múltiplo, na mira em 360° e na batalha final. De **Contra** (Konami, 1987) veio a lição do ritmo: inimigos em ondas legíveis e morte que devolve o jogador rapidamente à ação. Já **Celeste** (Maddy Makes Games, 2018) é a referência moderna de *game feel*: seu uso exemplar de coyote time e jump buffering – amplamente documentado pela comunidade – orientou a calibração das mesmas técnicas neste projeto. O diferencial autoral de Operação Resgate em relação a essas referências é o companheiro K9 com IA e progressão própria, mecânica incomum no subgênero.

3

PLANEJAMENTO DO PROJETO E MECÂNICAS PROPOSTAS

3.1 Do requisito ao escopo

O planejamento partiu dos requisitos mínimos do desafio – andar, correr, pular e escalar, com animações associadas – e os expandiu de forma deliberada para compor uma experiência com densidade de portfólio profissional. O Quadro 1 rastreia cada requisito do enunciado até a sua implementação no projeto.

Requisito do desafio	Implementação em Operação Resgate
Andar, correr, pular, escalar	Locomoção completa com inércia, corrida (Shift), pulo com coyote time e buffer, duplo pulo, escalada de escadas e, além do exigido: agachar, rolamento de esquiva com invulnerabilidade e deslize com salto encadeado (slide-jump)
Animações por ação	Máquina de estados com 15 estados visuais e folha de 18 poses do soldado; ciclos de caminhada/corrida do K9 com 6 quadros cada
3 a 5 fases com dificuldade crescente	5 fases com progressão planejada, culminando em batalha de chefe (Seção 4)
Obstáculos, coletáveis e desafios	12 tipos de coletáveis, caixas empurráveis e destrutíveis, portas interativas, elevadores, plataformas móveis/quebráveis/que desabam, fogo e barris explosivos
HUD com pontuação, vidas e itens	HUD completo: vida, vidas, pontuação, arma, munição (pente/reserva), retículo dinâmico e painéis do K9 (Seção 5)
Feedback visual e sonoro	Clarão de cano, projétil visível, cápsulas, tremor de câmera, faíscas, sons por ação e por superfície (Seções 5 e 8)
Trilha sonora e efeitos	Música por fase e mais de 25 efeitos sonoros gerenciados por serviço central

Quadro 1 – Rastreabilidade entre os requisitos do enunciado e o projeto. Fonte: o autor (2026).

3.2 O arsenal do soldado

O combate é sustentado por um sistema de armas **orientado a dados**: cada arma é uma ficha de atributos (dano, cadência, dispersão, alcance, capacidade, recarga, recuo e modo de disparo), o que torna o balanceamento uma tarefa de ajuste de números, sem tocar na lógica. A Tabela 1 resume o arsenal.

Arma	Dano	Cadência	Pente	Alcance	Modo
Pistola	22	4,5 tiros/s	12	16 u	Semiautomático
Fuzil	30	9 tiros/s	30	22 u	Automático
Espingarda	14 × 7 projéteis	1,4 tiros/s	6	12 u	Semiautomático
Metralhadora	16	13 tiros/s	60	20 u	Automático

Tabela 1 – Fichas de atributos das quatro armas do jogador. Fonte: o autor (2026).

Completam o combate as **granadas** de trajetória balística, o **golpe corpo a corpo** com faca – que economiza munição contra inimigos colados – e a **mira livre em 360°**: o projétil parte da boca do fuzil na direção exata apontada pelo

mouse, para frente, para cima, para baixo e nas diagonais, com o tronco do soldado inclinando até 24° para acompanhar visualmente a mira. Recarga temporizada, recuo físico e dispersão dinâmica (*bloom*, que cresce ao atirar e se recupera com o tempo) adicionam textura tática a cada disparo.

3.3 O companheiro K9-CYBER ALPHA

O K9 é um agente autônomo com máquina de estados própria – seguir, esperar, alerta, curando, furtivo e comemorando – implementada integralmente em código, sem NavMesh. Em alerta, escolhe entre três ataques conforme a distância do alvo: bote de mordida (curta), cuspe de fogo em cone (média) e laser da arma dorsal (longa). Fora do combate, cura o soldado gravemente ferido ao custo de energia própria. O companheiro acumula experiência pelos abates próximos, sobe até o nível 10 – ampliando vida, energia, velocidade e dano – e persiste sua evolução entre sessões pelo sistema de salvamento. Um LED de estado no corpo do cão comunica sua intenção por cor (azul seguindo, vermelho em alerta, verde curando, dourado comemorando), aplicando o princípio de comunicação não verbal ao design do personagem.

3.4 Controles

Ação	Entrada	Ação	Entrada
Mover	A/D ou setas	Atirar	Botão esq. do mouse / J
Correr	Shift	Mirar 360°	Movimento do mouse
Pular / duplo pulo	Espaço, W ou ↑	Recarregar	R
Agachar	S ou ↓	Trocar arma	Q, scroll ou 1-4
Rolamento (esquiva)	Ctrl esquerdo	Granada	G
Deslize	Correr + agachar	Corpo a corpo	V ou F
Escalar	W/S na escada	Interagir	E

Quadro 2 – Mapa de controles do jogador. Fonte: o autor (2026).

3.5 Ferramentas e decisão arquitetural

As ferramentas seguem o especificado no desafio: **Unity Engine** para o desenvolvimento e **C#** para a programação das mecânicas. Como decisão arquitetural central, adotou-se o paradigma **code-driven**: níveis, personagens, interface, materiais e efeitos são construídos por código em tempo de execução, a partir de uma camada declarativa de dados. Os benefícios são concretos:

reprodutibilidade total do projeto, versionamento limpo (o conteúdo é texto, não binários de cena), depuração determinística e a demonstração de domínio da API da engine em profundidade.

3.6 Metodologia e etapas de desenvolvimento

O desenvolvimento seguiu um processo **iterativo e incremental**, alinhado à prática profissional da indústria: em vez de construir o jogo inteiro em sequência linear, cada ciclo entregou uma versão jogável um pouco mais completa, avaliada em playtests e refinada no ciclo seguinte. O Quadro 3 resume as etapas percorridas e seus produtos.

Etapa	Atividades principais	Produto da etapa
1 · Concepção	Leitura do desafio, estudo de referências de mercado, definição de tema, narrativa e pilares (estratégia, ação, missão); esboço do GDD	Conceito e escopo aprovados
2 · Prototipagem	Locomoção e pulo com game feel calibrado; tiro básico; uma fase de teste cinza (greybox) para validar métricas	Protótipo jogável do núcleo
3 · Produção	Camada de dados dos níveis, cinco fases, inimigos e chefe, K9, arsenal completo, HUD, clima, áudio e save	Versão alfa com conteúdo completo
4 · Testes e ajuste	Playtests por fase, correção de defeitos de física e balística, balanceamento de armas e dificuldade	Versão beta estável
5 · Polimento e entrega	Efeitos, telas, feedback audiovisual final, exportação do executável, README e vídeo pitch	Entrega final e portfólio

Quadro 3 – Etapas da metodologia iterativa de desenvolvimento. Fonte: o autor (2026).

O registro do design foi mantido em um **GDD** (Game Design Document) enxuto – visão, mecânicas, fases e progressão – atualizado a cada iteração. Essa disciplina documental evitou a deriva de escopo, problema clássico de projetos de jogos, e serviu de fonte direta para este relatório e para o roteiro do vídeo pitch.

4

DESIGN DE NÍVEIS COM PROGRESSÃO DE DIFICULDADE

4.1 Arquitetura data-driven dos níveis

As cinco fases são descritas de forma **declarativa** em uma camada de dados que especifica, para cada nível: nome, objetivo, comprimento, plataformas (estáticas, móveis, elevatórias, quebráveis e que desabam), inimigos terrestres e aéreos com raio de patrulha, coletáveis, perigos, adereços interativos, checkpoints, clima (chuva e intensidade do vento), cor da iluminação e trilha musical. Um construtor de níveis lê essa descrição e monta a cena inteira em tempo de execução. Alterar o level design é, portanto, editar dados – nunca reescrever lógica.

4.2 As cinco fases

Fase	Ambiente	Novos elementos e desafios	Foco de aprendizado
1 · Favela Contaminada	Favela radioativa, luz esverdeada	Zumbis radioativos, mutantes, fogo e barris; plataformas baixas	Movimento, tiro, coleta e leitura de perigos
2 · Ruínas Urbanas	Ruínas sob chuva	Plataformas móveis e quebráveis, robô sentinela, drone, torre de vigia, primeiro portão e escada	Pulo duplo, escalada e mira vertical
3 · Campo Radioativo	Zona industrial	Helicóptero de combate, maior densidade de perigos ambientais	Mira aérea, uso de cobertura e granadas
4 · Linha de Frente	Rua devastada em guerra	Todos os inimigos combinados, elevadores e plataformas elevatórias	Domínio combinado de todas as habilidades
5 · Sala de Comando	Arena do chefe	Batalha contra o Comando Inimigo com padrões de ataque em fases	Esquiva, gestão de munição e cooperação com o K9

Quadro 4 – Progressão de dificuldade das cinco fases. Fonte: o autor (2026).

4.3 Princípios aplicados ao traçado

O traçado de cada fase aplica três princípios estudados na disciplina. Primeiro, **introduzir antes de combinar**: todo elemento novo aparece pela primeira vez em situação de baixo risco – a primeira plataforma móvel da Fase 2, por exemplo, cruza um vão sem inimigos – antes de ser combinado com ameaças. Segundo, **ritmo por alternância**: trechos de combate intenso são intercalados com respiros de plataforma pura e coleta, evitando fadiga. Terceiro, **falha barata**: checkpoints distribuídos a cada 20-25 unidades de percurso devolvem o jogador para perto do desafio que o derrotou, preservando o aprendizado da tentativa anterior em vez de puni-lo com repetição longa.

Os coletáveis cumprem função dupla: recompensa e sinalização. Medalhas marcam a rota principal; medkits, munição, baterias e itens de experiência premiam desvios que exigem manobras precisas; chaves e cartões conectam-se às portas e portões, criando micro-objetivos dentro do fluxo da fase. Quedas no vazio custam uma vida – o plano de queda é monitorado continuamente – e a extração ao final de cada fase é celebrada com animação de vitória, contagem de desempenho e comemoração do K9.

4.4 Métricas de dimensionamento

Todo o traçado é dimensionado por um conjunto de métricas derivadas da física do personagem – a régua citada na Seção 2.1. A Tabela 2 consolida os valores que garantem que **todo desafio do jogo é vencível**: nenhuma barreira é mais alta que o pulo, nenhum vão é mais largo que o alcance do salto em corrida e toda cobertura permite transposição com um pulo simples.

Métrica	Valor	Consequência no level design
Velocidade de caminhada	5,0 u/s	Base do ritmo de exploração
Velocidade de corrida	8,5 u/s	Define o alcance horizontal do salto longo
Gravidade	26 u/s ²	Queda firme, sem flutuação
Altura máxima do pulo	≈ 3,2 u	Teto para plataformas de um salto
Portas e portões	2,2 u	Transponíveis por cima ou abertos com E
Coberturas (sacos, cercas)	0,8 u	Protegem agachado e são puláveis
Espaçamento de checkpoints	20-25 u	Falha barata, ritmo preservado

Tabela 2 – Métricas de dimensionamento do level design. Fonte: o autor (2026).

5

INTERATIVIDADE, HUD E FEEDBACK

5.1 O circuito ação–resposta

Interatividade e feedback formam o circuito de comunicação entre o jogo e o jogador: toda ação relevante precisa produzir resposta perceptível e imediata, e todo estado importante do sistema precisa estar visível. A teoria de HUD estudada na disciplina distingue informação **persistente** (sempre exibida) de informação **contextual** (exibida no momento da ação) – e o projeto aplica as duas camadas.

5.2 Informação persistente: o HUD

O HUD exibe, de forma permanente e legível: barra de vida e vidas restantes; pontuação; arma equipada com munição do pente e da reserva; retículo de mira cuja abertura cresce ao atirar e ao correr – comunicando visualmente a dispersão dinâmica do sistema de armas – e os painéis do companheiro K9 com vida, energia, nível, barra de experiência e estado atual (seguindo, em guarda, alerta, curando). Banners de missão apresentam o objetivo no início de cada fase, e telas dedicadas tratam pausa, game over, conclusão de nível e vitória da campanha.

5.3 Informação contextual: o feedback de combate

Cada disparo produz uma cadeia completa de resposta: clarão na boca do cano, projétil visível que viaja até o ponto de impacto, cápsula ejetada com física própria (arco, quique e desaparecimento), som com assinatura da arma, tremor de câmera proporcional ao recuo, coice físico que empurra levemente o atirador nas armas pesadas e faíscas no ponto atingido. Coletar um item gera brilho, som e incremento imediato no HUD; receber dano dispara vibração de câmera, som de impacto e período de invulnerabilidade sinalizado; portas exibem luz vermelha fechadas e verde ao abrir; o pente vazio produz clique seco e recarga automática – avisando o jogador pelo ouvido antes que o perigo o alcance.

5.4 A câmera como instrumento de feedback

A câmera de acompanhamento usa suavização exponencial (SmoothDamp) com limites por fase, leve inclinação para dar profundidade ao 2.5D e **zoom automático contextual**: recua quando o jogador corre e nas batalhas de chefe – enquadrando a arena – e aproxima nos momentos calmos. O tremor de tela usa ruído Perlin com decaimento, produzindo vibração orgânica em explosões e dano. Elementos de clima – chuva, vento que balança adereços e iluminação própria por fase – reforçam a atmosfera sem jamais competir com a legibilidade do primeiro plano.

5.5 Usabilidade e acessibilidade de controle

Três decisões de usabilidade reduzem a barreira de entrada sem rebaixar o desafio. Primeiro, **mapeamento redundante**: toda ação essencial aceita mais de uma entrada (mover em A/D ou setas; pular em Espaço, W ou seta para cima; atirar no mouse ou em J), acomodando destros, canhotos e diferentes hábitos de teclado. Segundo, **redundância sensorial**: nenhuma informação crítica depende de um único canal – o dano é comunicado por som, tremor e HUD; o estado do K9, por cor de LED e painel; o pente vazio, por som e número. Terceiro, **pausa a**

qualquer momento, com retomada segura, respeitando o tempo do jogador. Essas escolhas seguem o princípio de que acessibilidade bem aplicada melhora a experiência de todos os jogadores, não apenas de quem dela depende.

6

ASSETS, SPRITES E ANIMAÇÕES NA UNITY

□ 6.1 Pipeline de sprites

A direção de arte combina sprites carregados da pasta Resources com elementos gerados proceduralmente por código (materiais, primitivas e efeitos). O soldado utiliza uma folha de **18 poses** – parado, ciclos de caminhada e corrida, pulo, queda, agachado, três poses de mira e tiro, escalada, recarga, arremesso de granada, golpe, ferido, vitória e duas poses de derrota – todas com **pivô nos pés**, garantia geométrica de que o personagem permanece apoiado no chão em qualquer pose e em qualquer rotação. O K9 possui ciclos próprios de caminhada e corrida com seis quadros cada, pose de descanso com sutil animação de respiração e espelhamento por escala para encarar cada lado.

6.2 Máquina de estados de animação

A animação é conduzida por uma **máquina de estados em C#** que mapeia quinze estados de jogo (parado, andando, correndo, pulando, caindo, agachado, escalando, atacando, rolando, corpo a corpo, recarregando, arremessando, ferido, vitória e derrota) para sequências de quadros com velocidades próprias – a corrida anima a 13 quadros/s, a escalada a 6. A abordagem foi escolhida em lugar do Animator Controller visual por oferecer controle determinístico, transições instantâneas e depuração trivial, preservando exatamente o mesmo conceito de estados e transições da ferramenta de animação da Unity – apenas expresso em código, coerente com a arquitetura do projeto.

6.3 Animação processual por cima dos quadros

Três camadas processuais enriquecem os quadros desenhados: a **inclinação de mira**, que gira o tronco do soldado em torno do pivô dos pés até $\pm 24^\circ$ acompanhando a direção do tiro; o **giro completo** do sprite durante o rolamento de esquiva; e o **bob de respiração** do K9 em repouso. Como o pivô está nos pés, todas as rotações mantêm o personagem plantado no solo – um exemplo direto de como uma decisão de pipeline (posição do pivô) habilita técnicas de animação a custo zero.

6.4 Cenário, luz e materiais

Os cenários usam **paralaxe** de fundo em múltiplas camadas com velocidades proporcionais à profundidade, vinhetas de clima (chuva com partículas, vento que balança adereços) e iluminação dinâmica com cor própria por fase – do verde tóxico da favela ao âmbar das ruínas. Materiais emissivos gerados por código dão brilho HDR a traçadores, lasers, coletáveis e sinalizações, e um utilitário central de materiais garante consistência visual entre todos os objetos instanciados. Adereços como carros destruídos, sacos de areia, cercas e caixas de munição completam a ambientação de guerra.

6.5 Origem e créditos dos assets

Em conformidade com o requisito de utilizar sprites originais ou de repositórios gratuitos com os devidos créditos, o projeto adota política explícita de proveniência: as folhas de sprites do soldado e do K9 foram produzidas para o projeto; primitivas, materiais, efeitos de partículas e elementos de interface são gerados proceduralmente por código, sem dependência de terceiros; e os efeitos sonoros são sintetizados pelo próprio sistema de áudio do jogo. Todo recurso e sua origem estão relacionados no README.md do repositório GitHub que acompanha a entrega, garantindo rastreabilidade e respeito à propriedade intelectual.

7

LÓGICA DE PROGRAMAÇÃO EM C# E O GAME LOOP

7.1 A estrutura do game loop

A Unity organiza a execução em um laço contínuo com etapas de responsabilidade distinta. Compreender – e respeitar – essa separação foi determinante para a estabilidade do projeto. A Figura 1 resume o ciclo aplicado a Operação Resgate.



Figura 1 – O game loop da Unity aplicado ao projeto. Fonte: o autor (2026).

- **Update (uma vez por quadro):** leitura de toda a entrada do jogador – teclas, mouse e mira em 360° – além de temporizadores de ações e avanço da animação. Entradas instantâneas, como o toque de pulo, são capturadas aqui e guardadas em *buffer*, pois o passo de física não ocorre em todos os quadros e lê-las lá perderia comandos.
- **FixedUpdate (passo de física, 50 Hz):** movimentação via *Rigidbody*, detecção de chão combinando contato físico real com *SphereCast* sob os pés, gravidade customizada e consumo do buffer de pulo – física estável e independente da taxa de quadros.
- **LateUpdate:** câmera e elementos que reagem após o movimento do mundo, evitando o tremor visual de seguir um alvo que ainda vai se mover no mesmo quadro.

□ 7.2 Arquitetura de componentes

O código é organizado em componentes coesos e de responsabilidade única: controle do jogador, sistema de armas, saúde, granadas, combate corpo a corpo, animação, IA de cada tipo de inimigo, IA do companheiro, construção de níveis, gerência de estado do jogo, áudio, salvamento e interface – mais de quarenta classes distribuídas em módulos *Player*, *Enemies*, *World*, *Core* e *UI*. O acoplamento entre sistemas é resolvido por um barramento de **eventos de combate**: ao disparar, o jogador publica o evento com posição e intensidade; inimigos próximos "ouvem" o tiro e reagem investigando a origem – sem que nenhuma classe conheça a outra. O padrão observer, estudado na disciplina, aparece aqui em sua forma mais útil (NYSTROM, 2014). A Figura 2 ilustra a organização.



Figura 2 – Arquitetura de módulos e barramento de eventos. Fonte: o autor (2026).

7.3 Organização do código-fonte

O Quadro 5 detalha a responsabilidade de cada módulo e exemplos de componentes que o compõem, evidenciando a aplicação dos princípios de coesão e responsabilidade única estudados em engenharia de software.

Módulo	Responsabilidade	Exemplos de componentes
Core	Estado global, dados e serviços transversais	Gerente do jogo, dados dos níveis, configurações, áudio, salvamento, eventos de combate, pool de efeitos
Player	Tudo o que pertence ao soldado	Controlador, sistema de armas, saúde, granadas, corpo a corpo, animador
Enemies	Inteligências artificiais hostis	Zumbi, mutante, robô sentinela, drone, torre, helicóptero, chefe
World	Construção e comportamento do mundo	Construtor de níveis, K9, plataformas, perigos, coletáveis, portas, elevadores, clima, checkpoints
UI	Interface e apresentação	HUD, retículo, painéis do K9, menus, telas de estado, câmera

Quadro 5 – Módulos do código-fonte e responsabilidades. Fonte: o autor (2026).

7.4 Técnicas de física e desempenho

A balística usa *SphereCast* com raio de 0,25 unidade, ordenação dos acertos por distância e filtragem de gatilhos – conciliando a precisão do raycast com a tolerância de acerto que o ritmo de ação exige. Um **object pool** de efeitos reutiliza instâncias de faíscas e clarões, eliminando alocações (e picos de garbage collector) durante o combate. O corpo do jogador emprega detecção contínua de colisão, material físico sem atrito – para não grudar em paredes – e limite de velocidade de despenetração, decisão que eliminou vibrações verticais causadas

pelo solucionador do PhysX. O sistema de salvamento persiste progresso de fases, recordes e a evolução do K9 entre sessões.

7.5 Orientação a dados

Dois subsistemas exemplificam a orientação a dados adotada: as armas (fichas de atributos, Tabela 1) e os níveis (descrições declarativas, Seção 4). Em ambos, o comportamento é genérico e o conteúdo é dado – o que reduz a superfície de erro, acelera o balanceamento e reflete a prática profissional de separar engine de conteúdo.

7.6 Testes, depuração e controle de versão

A qualidade foi sustentada por três práticas. **Playtests dirigidos por fase:** cada nível foi jogado repetidamente com objetivos de verificação específicos (todo pulo é possível? todo checkpoint salva? toda arma alcança o que a Tabela 1 promete?), e cada defeito encontrado gerou correção antes do ciclo seguinte. **Depuração de causa raiz:** os defeitos de física relatados na Seção 9 foram tratados até a causa no motor, não apenas até o sintoma – prática que evita regressões. **Controle de versão com Git:** o projeto é versionado e publicado em repositório GitHub com códigos-fonte organizados, assets, capturas de tela e README.md descrevendo jogo, mecânicas, controles e instruções de execução, conforme os entregáveis práticos do enunciado.

8 ESTRATÉGIAS DE ÁUDIO E EFEITOS SONOROS

8.1 Trilha sonora

A trilha sonora acompanha a curva emocional da campanha com música própria por fase: da tensão contida da Favela Contaminada ao tema de combate da Sala de Comando, passando por atmosferas de ruína e guerra aberta. A música é tratada como camada de estado – troca com a fase, silencia na derrota e celebra na vitória – mantendo coerência com o universo militar do jogo, como exige o enunciado.

8.2 Efeitos sonoros como informação

Os efeitos sonoros são gerenciados por um serviço central de áudio e cobrem todas as ações relevantes: disparo com assinatura própria por arma, clique de pente vazio, recarga, troca de arma, cápsulas tilintando no chão,

explosões, pulo e duplo pulo com sons distintos, dano, coleta por tipo de item, checkpoint, abertura de portas, latidos, rosnados e o latido feliz do K9, laser, chamas e a ambientação de chuva. O princípio regente é que o som **informa antes de ambientar**: o clique do pente vazio avisa o jogador pelo ouvido no exato momento em que os olhos estão no inimigo; o som de passos varia conforme a superfície – terra, pedra ou metal – ancorando o personagem no mundo.

8.3 Técnicas anti-fadiga e som posicional

Duas técnicas evitam a fadiga auditiva típica de jogos de ação. Primeiro, o gerenciador aplica **variação sutil de tom** entre repetições do mesmo efeito – nenhuma rajada soa como a anterior. Segundo, sons de cenário usam **reprodução posicionada no espaço**, com atenuação pela distância, de modo que uma explosão distante soa distante. O volume de cada categoria foi calibrado para que o feedback de combate nunca mascare os avisos críticos (dano recebido, pente vazio, latido de alerta do K9).

8.4 Implementação na engine

Na implementação, o serviço central de áudio mantém fontes (AudioSource) dedicadas por categoria – música, efeitos de combate, interface e ambiente – o que permite dosar cada camada de forma independente e interromper a categoria certa em pausas e transições. A variação anti-fadiga é obtida sorteando o *pitch* de cada reprodução dentro de uma faixa estreita em torno do tom original; o som posicional usa reprodução no ponto do evento com atenuação pela distância à câmera. Como os efeitos são sintetizados proceduralmente pelo próprio sistema, o jogo não depende de bancos de áudio externos – coerente com a política de proveniência de assets da Seção 6.5 – e o custo de memória da camada sonora permanece mínimo.

9

CONEXÃO ENTRE TEORIA E PRÁTICA

A construção de Operação Resgate materializou cada eixo teórico da disciplina em decisões mensuráveis de engenharia – e, mais importante, em **problemas reais diagnosticados e resolvidos**. Quatro casos ilustram esse ciclo completo de teoria, prática, falha e correção:

- **O pulo perdido.** A teoria do game loop ensina que Update e FixedUpdate correm em relógios diferentes. Na prática, ler o toque de pulo dentro do passo de física perdia comandos – o duplo pulo falhava de forma

intermitente. A solução foi o *jump buffering*: capturar a entrada no Update e consumi-la na física, exatamente como a literatura de game feel prescreve.

- **O personagem que tremia.** O soldado vibrava verticalmente parado no chão. O diagnóstico exigiu compreender o funcionamento interno do PhysX: reescrever as dimensões do colisor a cada passo destruía o cache de contatos; uma força de "colar no chão" excessiva re-penetrava o piso a cada ciclo; e o *SphereCast* nascendo dentro de uma parede devolvia falsos positivos de chão. Cada causa foi isolada e corrigida – só escrever no colisor quando há mudança real, zerar a velocidade vertical quando há contato físico e descartar acertos de distância nula.
- **O tiro torto.** A origem do disparo vinha de uma tabela de pontos por pose de animação; qualquer troca de quadro fazia o projétil nascer fora da arma. A correção aplicou geometria em lugar de tabelas: a boca do cano passou a ser calculada como ombro somado à direção da mira multiplicada pelo comprimento do fuzil – invariante, correta em 360° e imune a mudanças de arte.
- **A barreira ilegível.** Coberturas exibiam um visual maior do que o colisor real, porque o sprite era escalado pela largura e não pela altura: o jogador via um paredão intransponível onde a física oferecia um obstáculo baixo. A correção alinhou o que se vê ao que bloqueia – visual escalado pela altura e colisor idêntico ao desenho – aplicando o princípio de legibilidade da Seção 2: o jogador precisa poder ler o desafio para poder vencê-lo.

Além dos casos de correção, a teoria orientou o design desde o início: a curva de dificuldade das fases segue o princípio de introduzir-exercitar-combinar; o HUD separa informação persistente de contextual; o padrão observer desacopla o combate da IA; e a orientação a dados separa comportamento de conteúdo. O projeto demonstra, assim, o objetivo final da disciplina: transformar fundamentos em critério de decisão durante o desenvolvimento real.

10

CONSIDERAÇÕES FINAIS

Operação Resgate cumpre integralmente os requisitos do desafio – mecânicas completas de movimento com animações associadas, cinco fases de dificuldade crescente, HUD funcional, feedback visual e sonoro abrangente, arte e animação integradas à Unity e trilha sonora coerente com o universo do jogo – e os excede em profundidade: companheiro de combate com IA e progressão

persistente, arsenal orientado a dados com balística e mira em 360°, clima dinâmico, sistema de salvamento e uma arquitetura integralmente orientada a código com mais de quarenta componentes desacoplados por eventos.

Para o autor, o processo consolidou competências centrais da formação em Análise e Desenvolvimento de Sistemas: domínio da engine Unity e de sua API, programação orientada a objetos em C#, física de jogos aplicada, padrões de projeto (observer, object pool, máquinas de estado, orientação a dados), design de experiência do usuário e – talvez o aprendizado mais transferível – a depuração sistemática de problemas complexos, do sintoma visível à causa raiz no motor de física.

Como **trabalhos futuros**, o projeto está preparado para evoluir em quatro direções de baixo atrito, graças à arquitetura orientada a dados: novas fases e chefes intermediários (bastando descrever os dados dos níveis), suporte nativo a controle de videogame, modos de dificuldade selecionáveis e placares de pontuação, ampliando a vida útil do jogo como produto e como peça de demonstração técnica.

O resultado é uma experiência jogável completa, funcional e tecnicamente estruturada, acompanhada dos demais entregáveis previstos (jogo exportado, repositório documentado e vídeo pitch), e pronta para compor o portfólio profissional do autor como evidência de capacidade técnica e criativa em desenvolvimento de jogos.

REFERÊNCIAS

BRACKEYS. **How to make a 2D platformer** – Unity tutorial. Disponível em: <https://www.youtube.com/watch?v=dwcT-Dch0bA>. Acesso em: 14 jul. 2026.

FULLERTON, Tracy. **Game design workshop**: a playcentric approach to creating innovative games. 4. ed. Boca Raton: CRC Press, 2018.

GAME DEVELOPMENT. **Material didático da disciplina.** São Paulo: UniFECAF, 2026.

NYSTROM, Robert. **Game programming patterns.** [S. l.]: Genever Benning, 2014.

ROGERS, Scott. **Level up:** um guia para o design de grandes jogos. São Paulo: Blucher, 2013.

SHELL, Jesse. **A arte de game design:** o livro original. Rio de Janeiro: Elsevier, 2011.

SENTRY. **Unity tutorial:** developing your first Unity game – part 1. Disponível em: <https://blog.sentry.io/unity-tutorial-developing-your-first-unity-game-part-1/>. Acesso em: 14 jul. 2026.

SWINK, Steve. **Game feel:** a game designer's guide to virtual sensation. Boca Raton: CRC Press, 2009.

UNITY TECHNOLOGIES. **Unity Learn:** get started. Disponível em: <https://unity.com/pt/learn/get-started>. Acesso em: 14 jul. 2026.

UNITY TECHNOLOGIES. **Unity Manual:** MonoBehaviour, Physics e Scripting API. Disponível em: <https://docs.unity3d.com>. Acesso em: 14 jul. 2026.